HABALGO SYSTEM PROGRESS DOCUMENTATION

Group No. 6

Members	Roles
Villareal, Mae	Project Manager
Barte, Roxanne	UI/UX
Royo, Jeana	Back-end
Remoquillo, Michelin	front-end
Balansay, Bohdana	Quality Assurance
Gumahin, Jewey Kristine	Quality Assurance

Repository link: https://github.com/jeanaroyo/HabalGo.git
Project Tracker: [BSIT 3-1] - PROJECT TRACKER.xlsx

System overview

Purpose of the Project
HabalGo is a centralized relational database management system running on PostgreSQL that manages customers' bookings, drivers allocation, vehicle details, deliveries, and payments for motorcycle/vehicle ride-sharing.

Scope
Manages transaction processing (OLTP) transactions such as creating, modifying, cancelling bookings, allocation of vehicles to drivers, payments and customer reviews.

Main functions

1. User and driver account profile registration
2. Vehicle registration and assignment management
3. Ride and delivery request booking (Picking up and drop off tracking)
4. Payment processing and transaction recording
5. Driver designation and ride status tracking
6. Rating and feedback submission

User Roles

Application Role	Responsibilities in the Application	Functions and Data Access	Intended Database Role (Example)
System Administrator	Manages system-wide	Create, read, update, and delete across all	app_admin_grp

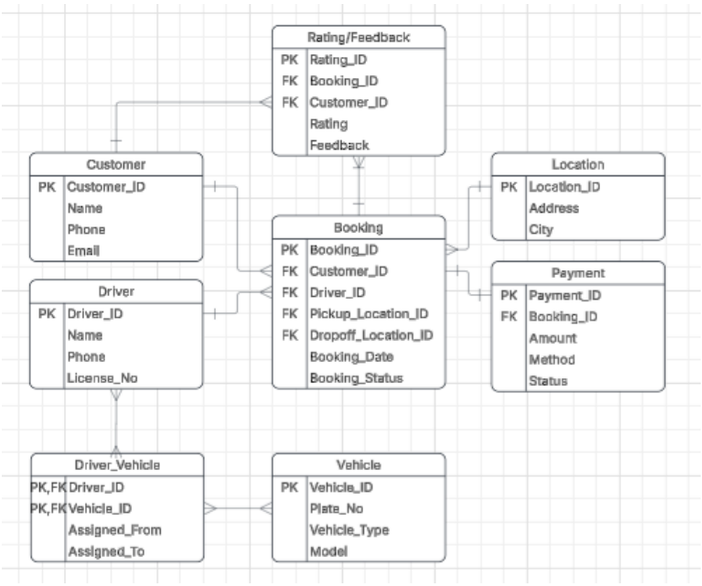
	operations, maintains master registries, and monitors application health.	tables (customer, driver, vehicle, driver_vehicle, location, booking, payment, rating_feedback); read on system audit reports.	
Operations Staff	Oversees daily dispatching, manages vehicle assignments, updates ride/delivery statuses, and handles customer support.	Create and update on driver_vehicle assignments and booking statuses; read on customer, driver, vehicle, location, payment, and rating_feedback.	app_operations_grp
Passenger / Customer	Registers profile, requests ride or delivery bookings, processes payments, and submits feedback for completed trips.	Create, read, and update on own customer profile; create and read on booking and payment; create on rating_feedback; read on location.	app_customer_grp
Driver	Updates active vehicle association, accepts/manages ride bookings, updates trip progress, and tracks earnings/feedback.	Read and update on own driver profile and driver_vehicle assignments; read and update booking status; read on vehicle, location, payment, and rating_feedback.	app_driver_grp

Development progress log

Development activity	**Responsible member/s**	**status**
Application planning	All members	Completed
Database creation and configuration	Villareal, Mae; Barte, Roxanne; Gumahin, Jewey	Completed
ERD development	Barte, Roxanne	Completed
UI/UX improvement	Royo, Jeana	Completed
System testing	Barte, Roxanne	Completed
Technical Docs	Remoquillo, Michelin; Balansay, Bohdana	Completed

Database implementation

ERD



SQL scripts used

```sql
CREATE SCHEMA IF NOT EXISTS habalgo;

CREATE TABLE IF NOT EXISTS habalgo.customer (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS habalgo.driver (
    driver_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    license_no VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS habalgo.vehicle (
    vehicle_id SERIAL PRIMARY KEY,
    plate_no VARCHAR(20) NOT NULL UNIQUE,
    vehicle_type VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL
);
```

```sql
CREATE TABLE IF NOT EXISTS habalgo.driver_vehicle (
    driver_id INT NOT NULL REFERENCES habalgo.driver(driver_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    vehicle_id INT NOT NULL REFERENCES habalgo.vehicle(vehicle_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    assigned_from TIMESTAMP NOT NULL,
    assigned_to TIMESTAMP NULL,
    PRIMARY KEY (driver_id, vehicle_id),
    CONSTRAINT ck_assignment_dates CHECK (assigned_to IS NULL OR assigned_to >= assigned_from)
);

CREATE TABLE IF NOT EXISTS habalgo.location (
    location_id SERIAL PRIMARY KEY,
    address VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS habalgo.booking (
    booking_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES habalgo.customer(customer_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    driver_id INT NULL REFERENCES habalgo.driver(driver_id) ON UPDATE CASCADE ON DELETE SET NULL,
    pickup_location_id INT NOT NULL REFERENCES habalgo.location(location_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    dropoff_location_id INT NOT NULL REFERENCES habalgo.location(location_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    booking_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    booking_status VARCHAR(50) NOT NULL DEFAULT 'Pending',
    CONSTRAINT ck_booking_status CHECK (booking_status IN ('Pending', 'Confirmed', 'In Progress', 'Completed', 'Cancelled'))
);
```

```sql
CREATE TABLE IF NOT EXISTS habalgo.payment (
    payment_id SERIAL PRIMARY KEY,
    booking_id INT NOT NULL REFERENCES habalgo.booking(booking_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),
    method VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'Pending'
);

CREATE TABLE IF NOT EXISTS habalgo.rating_feedback (
    rating_id SERIAL PRIMARY KEY,
    booking_id INT NOT NULL REFERENCES habalgo.booking(booking_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    customer_id INT NOT NULL REFERENCES habalgo.customer(customer_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    feedback TEXT NULL,
    CONSTRAINT uq_rating_booking_customer UNIQUE (booking_id, customer_id)
);

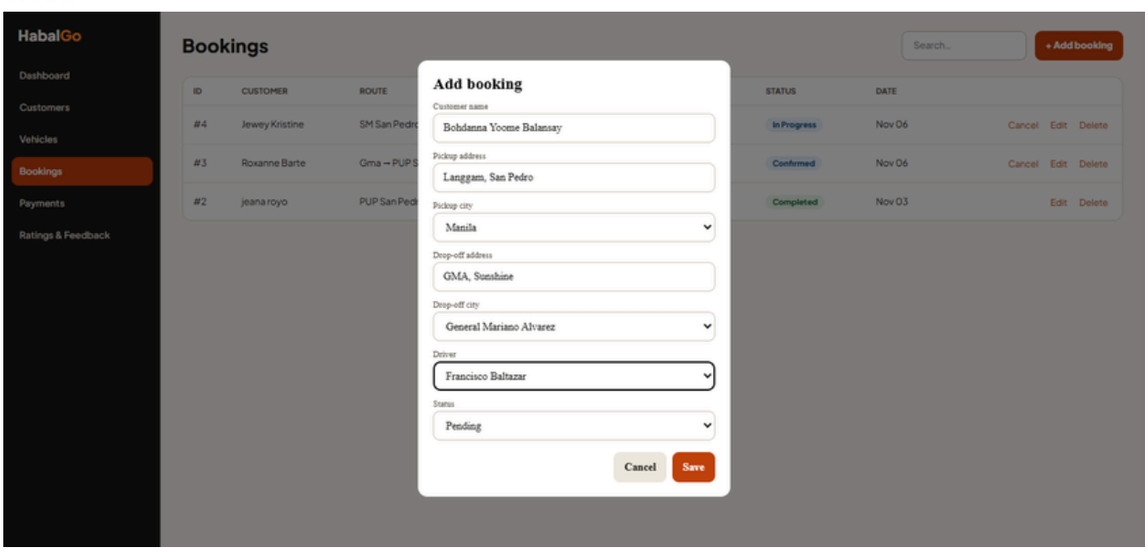
CREATE INDEX IF NOT EXISTS idx_booking_customer ON habalgo.booking(customer_id);
CREATE INDEX IF NOT EXISTS idx_booking_driver ON habalgo.booking(driver_id);
CREATE INDEX IF NOT EXISTS idx_booking_date ON habalgo.booking(booking_date);
CREATE INDEX IF NOT EXISTS idx_payment_booking ON habalgo.payment(booking_id);
CREATE INDEX IF NOT EXISTS idx_rating_booking ON habalgo.rating_feedback(booking_id);
```

seed data

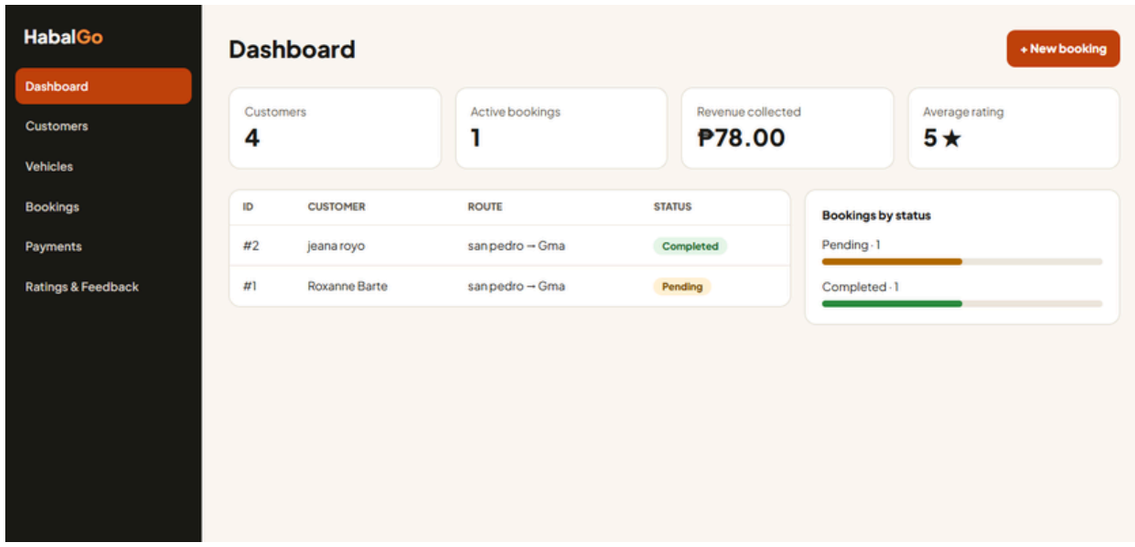
Entity	Seeded Data
Customer	6 registered customer profiles with full names, contact numbers and verified email account
Driver	3 active drivers with valid license numbers
Vehicle	3 registered motorcycles with distinct license plate numbers
Driver vehicle	3 driver-vehicle pairing records assigning each active driver to a motorcycle with valid start timestamps
location	4 pickup and drop-off locations across san pedro and GMA
booking	3 sample trip bookings testing various statuses: 1 completed, 1 in progress, and 1 pending
payment	2 payment transaction records capturing both E-wallet and cash tenders to validate financial logs
rating/ feedback	1 completed customer rating (5 stars) and feedback comment that is connected to a completed booking to test feedback submission logic

Application evidence

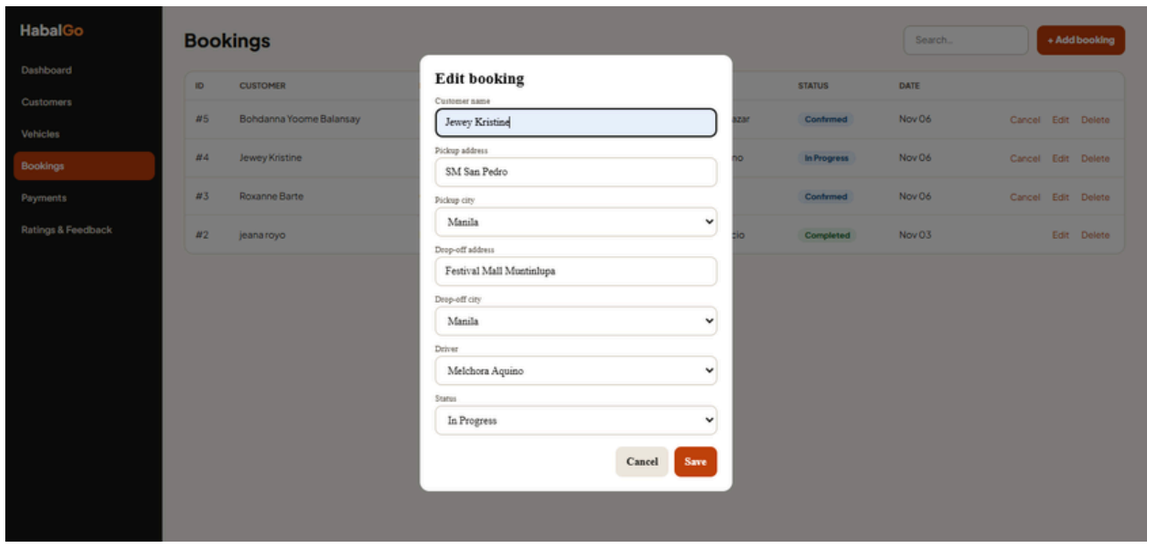
Create



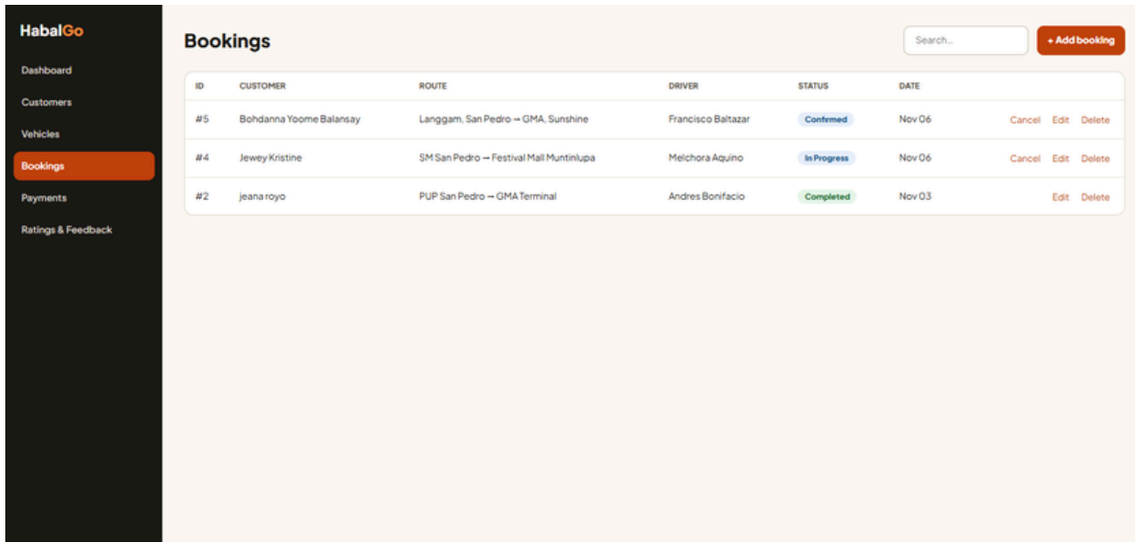
Read



Update



Delete



Setup and configuration

Clone the Repository
git clone https://github.com/jeanaroyo/HabalGo.git
cd HabalGo

Environment Configuration
Copy .env.example to create your local .env configuration file:

cp .env.example .env
Open .env and configure your local PostgreSQL database credentials:

DB_HOST=localhost
DB_PORT=5432
DB_NAME=grp00_habalgodb
DB_SCHEMA=habalgo
DB_USER=postgres
DB_PASS=your_postgres_password
Quick Reminder: Make sure to enable your pdo_pgsql extension! If not then do this steps:

1. Open XAMPP Control Panel
2. Click Apache Config
3. Go to PHP.ini
4. Search for extension=pdo_pgsql and extension=pgsql and remove the ;
5. Save and close

Initialize the PostgreSQL Database
1. Create Database:

CREATE DATABASE grp00_habalgodb;

2. Connect to the Database and Run Schema:

psql -U postgres -d grp00_habalgodb -f schema.sql

3. Load Seed / Sample Data:

psql -U postgres -d grp00_habalgodb -f "sample data.sql"

Run the Application
Option A: Using PHP Built-in Server (Easiest)
Run the following command in the project root folder:
php -S localhost:8000
Open your browser and navigate to: http://localhost:8000

Option B: Using XAMPP / htdocs
1. Move the HabalGo folder inside your xampp/htdocs/ directory.
2. Start Apache in XAMPP Control Panel.
3. Open your browser and navigate to: http://localhost/HabalGo

Issues and resolutions

Problems met during development and setup, and how they were solved.

During the process of working, the group encountered certain issues related to connecting the PostgreSQL database structure to VSCode extensions such as SQLTools and PostgreSQL

Explorer. The problem was caused by incorrect port numbers and server login credentials. It turned out that there were problems with the permissions of the service.

In order to overcome the mentioned difficulties, the group searched the Internet and watched several video tutorials as well as read instructions. They checked whether the connection strings were correct and verified the port number which is usually 5432 as well as the correct username and password of the database.

AI Usage declaration

AI tools used:

- Gemini/ Claude

Claude and Gemini were used to assist in building the database system and architecture. The group reviewed, modified, tested, and validated all generated outputs. The database requirements, business rules, ERD, schema, constraints, and architecture were ultimately managed and verified by the group.